Practical:03	Implementation of max-heap sort algorithm
__/___/2026	

Objective: To implement the Max-Heap Sort algorithm using C++ and understand its working by constructing a Max Heap, repeatedly extracting the largest element, and arranging the elements in ascending order while analyzing its time and space complexity.

Program / Code:

```
#include <iostream>
using namespace std;
// Function to maintain Max Heap
void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;
    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
// Heap Sort Function
void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // Extract elements one by one
```

```
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);  // Move largest element to end
        heapify(arr, i, 0);    // Restore heap property
    }
}
// Print Array
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
int main()
{
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original Array: ";
    printArray(arr, n);
    heapSort(arr, n);
    cout << "\nSorted Array: ";
    printArray(arr, n);
    return 0;
}
```

Output:

```
Original Array: 12 11 13 5 6 7 9 8
Sorted Array ( Descending ): 13 12 11 9 8 7 6 5
```

Analysis:

1. **Treat the array as a complete binary tree.**
2. **Convert the array into a Max Heap using heapify().**
3. **The root always stores the maximum element.**
4. **Swap the root with the last unsorted element.**
5. **Reduce the heap size by one.**

6. **Restore the Max Heap property using heapify().**
7. **Repeat until all elements are sorted.**

Build a Max Heap

The first loop converts the array into a Max Heap, where every parent node is greater than its children.

for (int i = n / 2 - 1; i >= 0; i--)

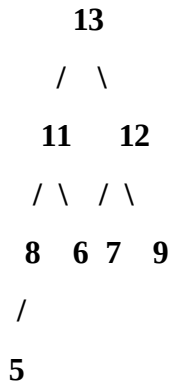
heapify(arr, n, i);

Why start from n/2 - 1?

Leaf nodes already satisfy the heap property. Only internal nodes need checking.

Original Array: 12 11 13 5 6 7 9 8

Max Heap



Time Complexity Analysis

Operation	**Complexity**
Build Max Heap	**O(n)**
Heapify (Single Call)	**O(log n)**
Heapify repeated for all elements	**O(n log n)**
Overall Heap Sort	**O(n log n)**

Conclusion:

The Max Heap Sort algorithm was successfully implemented and tested. The input array was first converted into a Max Heap, and then the largest element was repeatedly moved to its correct position until the array was completely sorted. This practical helped in understanding the working of the heapify process, heap construction, and the Heap Sort algorithm. Heap Sort is an efficient in-place sorting algorithm with a time complexity of O(n log n).